Table for Four — Reasoning, Memory & Tools

CMU AI Agent Certification — Capstone · Week 2 Submission
Author: Manish Bhatt

> Week 2 design write-up: the agent's reasoning loop, memory needs, and tool use, and why they beat a prompt-only approach. The reasoning here is already realized in the working orchestrator (Milestone 3), but the focus below is on *design rationale*, not code. The design will keep evolving as later modules add the human-approval gate, governance, and enhancements.

1. The reasoning loop

The agent uses a **structured Reason → Act → Observe loop**, the same core cycle as ReAct, but with the control flow made **explicit** as a state machine rather than left to free-form, model-chosen tool calls. Each step reasons about the current state, acts by calling a tool, observes the structured result, and that observation decides the next step.

The cycle, step by step:

- **Reason (interpret & plan).** The request — *"Italian, near Midtown, 4 people, Friday 7pm, one guest is gluten-free"* — is parsed into structured constraints (cuisine, party size, date/time, dietary, budget). This plan drives every downstream action.
- **Act (call a tool).** The agent searches for restaurants using those constraints, then retrieves matching perks, then checks availability.
- **Observe (read the result).** Each tool returns a structured observation: how many candidates matched, which perks fit, whether a time slot is open.
- **Decide the next action from the observation.** This is where reasoning guides action, at two explicit decision points:
  - **After search — refine or proceed.** If the observation is *"no candidates,"* the agent reasons that its constraints were too strict, **relaxes one** (drops a rating floor, raises the price ceiling, drops the cuisine filter) and **loops back to search**. If candidates exist, it proceeds to ranking. A **max-iteration guard** bounds the loop so it can never spin.
  - **Before booking — the human gate.** The agent proposes a specific reservation; an approval observation decides whether to **act (book)** or **stop**. Booking is the one irreversible step, so it is never taken without this checkpoint.
- **Act on the real world & record.** On approval the agent books and writes an audit entry.

How observations influence subsequent decisions is the heart of the loop: the *search result count* drives the refine-retry decision; the *availability result* determines which time slot is proposed; the *approval* determines whether the irreversible booking fires. Nothing downstream is decided in advance — each choice is a function of what the previous tool actually returned.

Why structured over free-form ReAct: the task is transactional. A predictable, inspectable flow lets us *guarantee* the irreversible action is always gated and the loop always terminates — safety properties that a free-form tool-calling agent cannot promise.

2. Memory

The task needs **both short-term and long-term memory**, in three tiers with distinct jobs.

Short-term / working memory — required for the core task. A single request is *multi-step*: ranking depends on the search results, the proposal depends on the ranking, the booking depends on the proposal. The agent therefore carries an evolving working state (parsed constraints → candidates → matched perks → chosen option → pending booking) across every step. It is needed the moment the task spans more than one tool call — without it, the agent could not carry search results forward to the booking step. This working state is also **persisted per conversation**, which is what lets the human-approval gate **pause and later resume** an in-flight booking.

Long-term memory — required to be a *concierge*, not a one-shot search. A member profile holds standing facts: dietary defaults (*"always gluten-free"*), cuisine and price preferences, and past bookings. It is *read* when a new request is parsed (to seed constraints, so a returning user need not re-state them) and *written* after a successful booking. It is needed **across sessions** — its whole value is that the agent remembers you next time and honors a standing dietary need or avoids re-booking the same place.

Episodic / audit memory — required for accountability. An append-only record of each run's tool calls, data sources, and approvals. It is needed whenever the system must *explain after the fact* why it acted and on what data, and it doubles as booking history.

Summary: short-term memory is essential to complete a single multi-step booking; long-term memory is what makes repeat use personal and safe.

3. External tools

A language model alone cannot know real-world facts, retrieve unstructured knowledge it wasn't trained on, or change the state of an external system. The agent uses three tools, each addressing a specific limitation:

Tool	Limitation it addresses
Restaurant search	**Grounding + retrieval** — real, current restaurants, ratings, prices, hours, and locations that are not in the model's parameters.
Perks retrieval (semantic search)	**Retrieval** over an unstructured offers store the model was never trained on.
Booking	**Action** — effecting a real change (a reservation) in an external system, which a model can describe but not perform.

A task where tool use is *necessary*: *"Find Italian restaurants near Midtown with a rating of at least 4.5, then reserve a table for four."* No amount of prompting lets the model answer this correctly on its own — it cannot know which restaurants currently exist, what their real ratings are, or whether a table is free, and it certainly cannot *make* the reservation. The **search tool grounds** the recommendation in real data, and the **booking tool acts** to create a verifiable reservation with a confirmation id. The tools supply exactly the two things the model lacks: *current facts* and the *ability to act*.

4. Why this beats a prompt-only approach

A prompt-only model is fluent but ungrounded, stateless, and unable to act. Our additions resolve concrete failure modes.

The specific failure mode this design resolves — the hallucinated booking. Ask a prompt-only model to *"book me a gluten-free-friendly Italian place in Midtown for Friday,"* and it will confidently invent a plausible-sounding restaurant, a fake address and phone number, and "confirm" a reservation that was never made. The user is left with a fabrication that fails at the worst possible moment — when they show up. Our design makes each part of that answer *real and verifiable*:

- **Grounding (search tool)** — the restaurant is a real one returned by a live data source, not an invention.
- **Action + verification (booking tool)** — the reservation is actually created and returns a confirmation id that can be looked up; a "confirmation" is no longer just reassuring text.
- **Provenance labeling** — every result is tagged with its data source, so mock or synthetic data is never passed off as live.
- **Human gate** — a person approves the irreversible action, so a wrong inference can't silently become a wrong booking.

Other prompt-only failure modes the design also removes: **staleness** (availability is checked live, not guessed), **forgetting** (working memory carries the multi-step plan; profile memory carries the user across sessions), and **unsafe autonomy** (the gated loop prevents unapproved actions). The reasoning loop adds one more: **brittle over-constrained queries** — a prompt-only system that finds nothing simply fails, whereas the refine-retry loop reasons about *why* it found nothing and relaxes the query.

5. Scope of this write-up

This document focuses on **design reasoning** — the shape of the loop, the roles of memory, and the limitations tools address — rather than implementation detail. The design is intentionally a living one: upcoming modules add the full human-in-the-loop approval gate and governance/audit trail, and later enhancements (member preferences, model comparison, web enrichment, illustrative imagery). The reasoning above is the stable core those additions will build on.
